BARBERPRO

Documentação Técnica do Projeto

Sistema de Gestão de Barbearia — Web Full-Stack

Gabriel da Silva Coelho | Frederico da Silva Kunert | Arthur Boaventura Riesco

PUC Goiás — Análise e Desenvolvimento de Sistemas

1. Identificação do Projeto

Nome do Projeto: BarberPro

Repositório: https://github.com/GabrielDaSilvaCoelho/BarbeariaTerminal

Protótipo Figma: https://www.figma.com/design/dF1zbDsM0UU1dAzf8m0vzO/figma-integrador

Integrantes: Gabriel da Silva Coelho, Frederico da Silva Kunert, Arthur Boaventura Riesco.

Curso: Análise e Desenvolvimento de Sistemas — PUC Goiás

Descrição da Proposta

O BarberPro é uma aplicação web full-stack desenvolvida para digitalizar e otimizar a gestão de serviços e agendamentos de uma barbearia. O sistema atende dois perfis principais de usuário — administrador/barbeiro e cliente — oferecendo controle de agenda, cadastro de serviços e gerenciamento de atendimentos de forma digital, organizada e responsiva.

Problema que o Sistema Resolve

Em barbearias que ainda utilizam processos manuais para marcação de horários, é comum ocorrer:

- Conflitos de agendamento por falta de controle centralizado
- Dificuldade no controle e visualização da agenda
- Falta de padronização nos serviços cadastrados
- Dificuldade de acesso do cliente ao próprio histórico de atendimentos
- Pouca organização administrativa geral

O BarberPro foi criado para resolver esse cenário por meio de uma aplicação web simples, funcional e acessível.

2. Diagramas C4

Os diagramas C4 descrevem a arquitetura do sistema em dois níveis de abstração: Contexto (visão macro dos atores e sistemas) e Container (visão interna dos componentes do sistema).

2.1 Diagrama de Contexto (C4 — Nível 1)

O diagrama de contexto apresenta o sistema BarberPro em relação aos seus usuários e sistemas externos. Dois atores interagem com o sistema: o Cliente, que agenda serviços e acompanha status, e o Administrador da Barbearia, que gerencia agenda, serviços e atendimentos. O sistema se integra com um Serviço de Notificação externo para envio de mensagens de eventos.

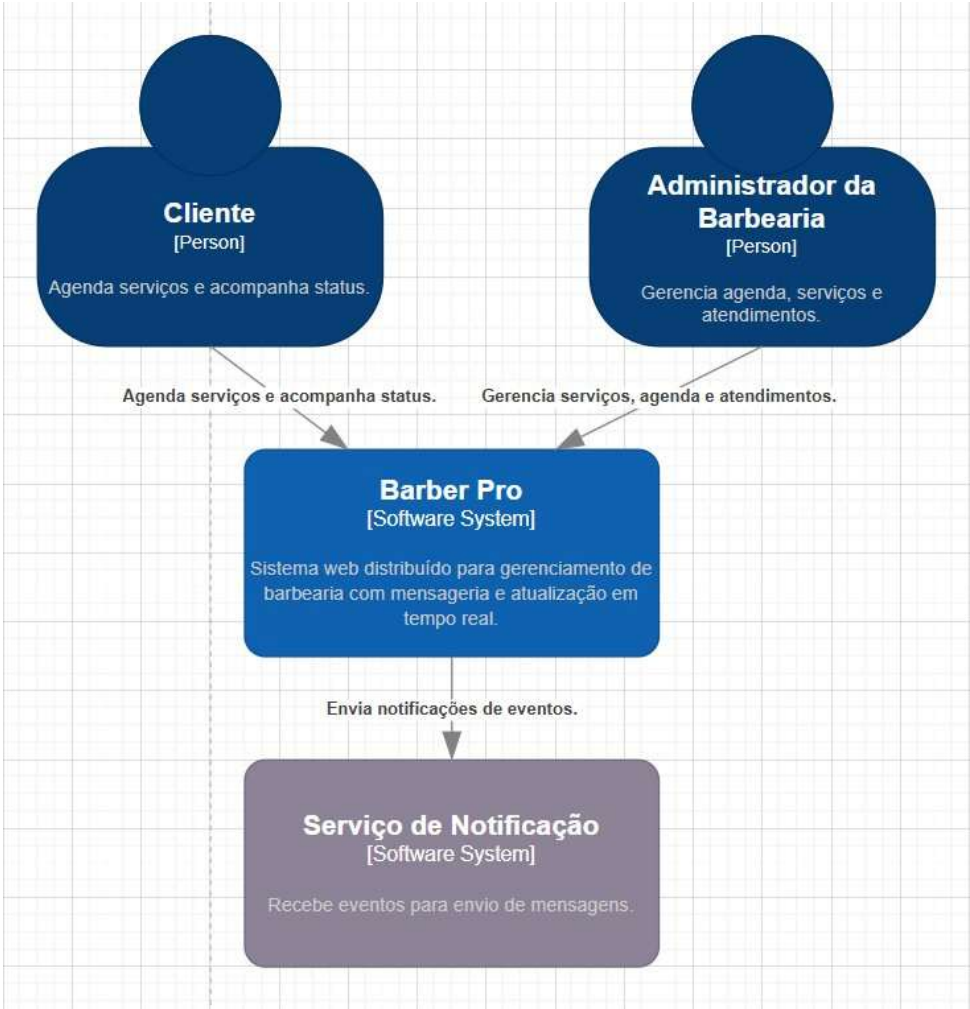

Figura 1 — Diagrama C4 de Contexto do sistema BarberPro

2.2 Diagrama de Container (C4 — Nível 2)

O diagrama de container detalha os componentes internos do sistema BarberPro. A arquitetura é composta por seis containers: Frontend Web (React), API Principal (SpringBoot), Broker de Mensagens (RabbitMQ), Banco de Dados (PostgreSQL), Worker (SpringBoot) e WebSocket (STOMP). O fluxo de dados parte do frontend, passa pela API, publica eventos no broker, que são consumidos pelo Worker e refletidos em tempo real via WebSocket.

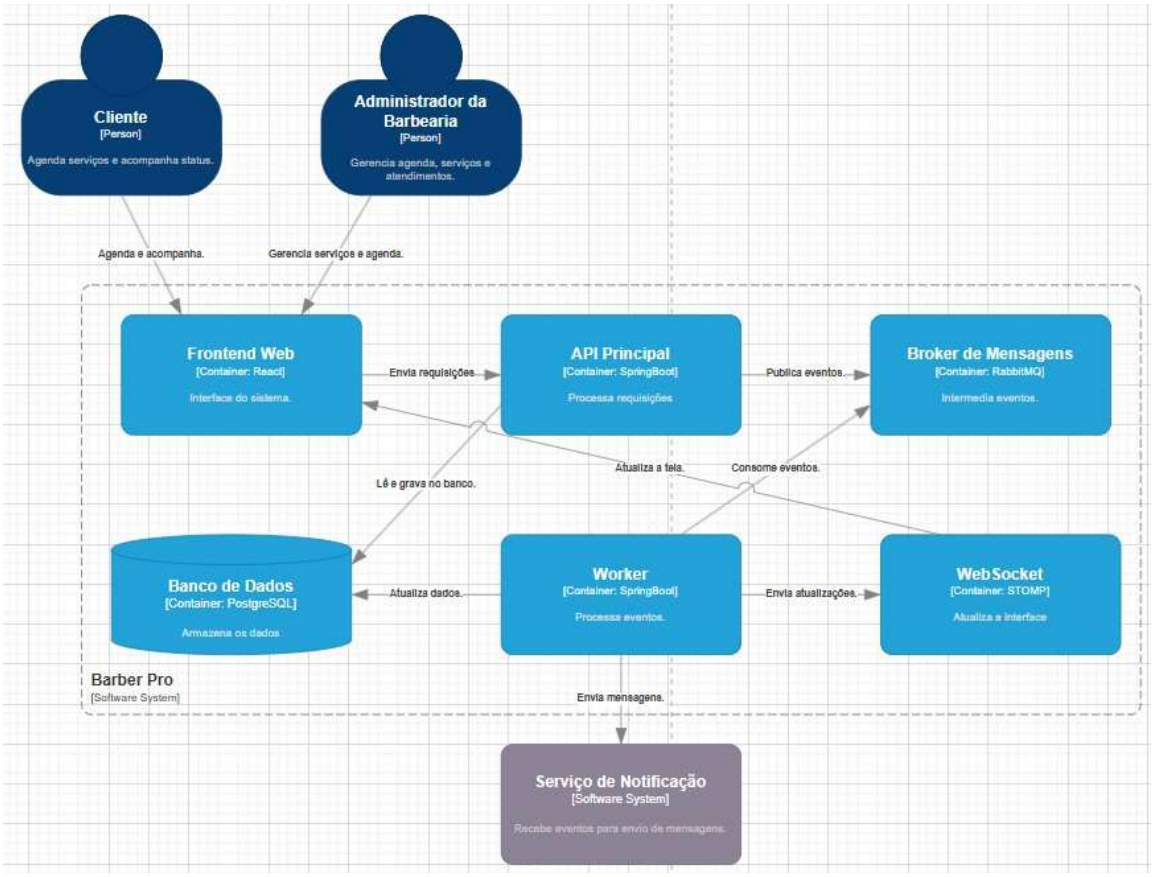

Figura 2 — Diagrama C4 de Container do sistema BarberPro

3. Diagrama de Classes do Domínio

O domínio do BarberPro é composto por três entidades principais que se relacionam para suportar todo o fluxo de agendamentos da barbearia.

3.1 Diagrama Visual

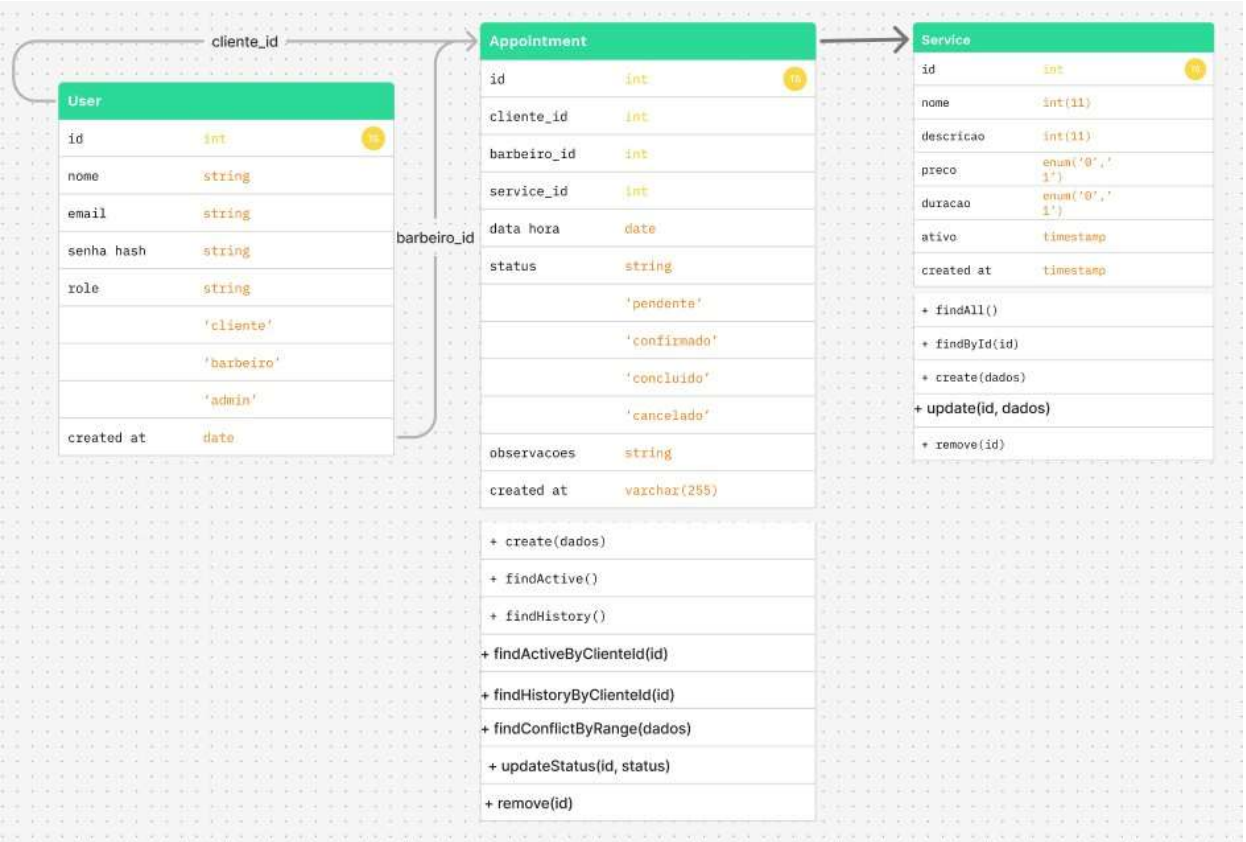

Figura 3 — Diagrama de Classes do domínio BarberPro

3.2 Entidades e Atributos

User

Atributo	Tipo	Descrição
id	int (PK)	Identificador único
nome	string	Nome completo do usuário
email	string (UNIQUE)	E-mail de acesso
senha_hash	string	Senha criptografada com bcrypt
role	string	'cliente' | 'barbeiro' | 'admin'
created_at	Date	Data de cadastro

Service

Atributo	Tipo	Descrição

id	int (PK)	Identificador único
nome	string	Nome do serviço
descricao	string	Descrição opcional
preco	decimal(10,2)	Preço em reais
duracao_min	int	Duração em minutos
ativo	boolean	Se o serviço está disponível
created_at	Date	Data de criação

Appointment

Atributo	Tipo	Descrição
id	int (PK)	Identificador único
cliente_id	int (FK)	Referência ao User (cliente)
barbeiro_id	int (FK)	Referência ao User (barbeiro)
service_id	int (FK)	Referência ao Service
data_hora	Date	Data e hora do agendamento
status	string	'pendente' | 'confirmado' | 'concluido' | 'cancelado'
observacoes	string	Observações opcionais
created_at	Date	Data de criação

3.3 Relacionamentos

Entidade A	Cardinalidade	Entidade B	Via
User	1 ——— *	Appointment	cliente_id
User	1 ——— *	Appointment	barbeiro_id
Service	1 ——— *	Appointment	service_id

3.4 Mapeamento de Camadas

Camada	Classe/Arquivo	Dependência
Repository	UserRepository	tabela users (PostgreSQL)
Repository	ServiceRepository	tabela services (PostgreSQL)
Repository	AppointmentRepository	tabela appointments (PostgreSQL)
Service	AuthService	UserRepository
Service	ServiceService	ServiceRepository

Service	AppointmentService	AppointmentRepository + ServiceRepository
Controller	AuthController	AuthService
Controller	ServiceController	ServiceService
Controller	AppointmentController	AppointmentService
Middleware	authMiddleware	JWT (jsonwebtoken)
Middleware	roleMiddleware	req.user.role
Middleware	errorMiddleware	statusCode padronizado

4. Indicação de Padrões e Arquitetura

4.1 Arquitetura em Camadas (Layered Architecture)

O sistema foi estruturado em arquitetura em camadas com separação clara de responsabilidades entre cinco níveis distintos:

Camada	Responsabilidade	Arquivos Principais
Frontend	Interface do usuário — chamadas HTTP à API REST	login.html, dashboard-admin.html, dashboard-cliente.html
Routes	Define os endpoints e aplica middlewares	auth.routes.js, services.routes.js, appointments.routes.js
Controllers	Recebe req/res, chama o service e retorna resposta HTTP	auth.controller.js, service.controller.js, appointment.controller.js
Services	Contém as regras de negócio e validações	auth.service.js, service.service.js, appointment.service.js
Repositories	Acesso ao banco via queries SQL parametrizadas	user.repository.js, service.repository.js, appointment.repository.js

4.2 Design Patterns Aplicados

Pattern 1 — Middleware (Chain of Responsibility)

Categoria: Comportamental

Problema: Era necessário proteger rotas sem repetir código de autenticação e autorização em todos os controllers.

- authMiddleware — verifica e decodifica o JWT, injeta req.user na requisição
- roleMiddleware — verifica se req.user.role possui permissão para acessar a rota
- errorMiddleware — captura erros lançados em qualquer camada e retorna resposta HTTP padronizada

Benefícios: Reutilização, segurança centralizada, fácil manutenção e desacoplamento.

Pattern 2 — Facade (Fachada)

Categoria: Estrutural

Problema: O frontend não pode acessar diretamente o banco, os services ou as validações internas.

- AuthController — fachada para registro, login e consulta de perfil (/me)
- ServiceController — fachada para CRUD completo de serviços
- AppointmentController — fachada para agendamentos e mudança de status

Benefícios: Simplificação do acesso, menor acoplamento, ponto de entrada único por domínio.

4.3 Princípios SOLID Aplicados

S — Single Responsibility Principle (SRP)

Cada camada e arquivo possui uma única responsabilidade bem definida. Routes definem apenas endpoints, controllers tratam apenas req/res, services contêm apenas regras de negócio e repositories executam apenas queries ao banco.

Módulo	Única Responsabilidade
Routes	Definir endpoints e aplicar middlewares
Controllers	Receber req/res e delegar ao service
Services	Regras de negócio e validações
Repositories	Acesso ao banco de dados
Middlewares	Autenticação, autorização e erros

D — Dependency Inversion Principle (DIP)

Camadas superiores dependem de abstrações (services), não de detalhes de implementação (banco de dados). O AppointmentController depende de AppointmentService, que por sua vez depende de AppointmentRepository, isolando completamente o PostgreSQL das camadas superiores.

4.4 Tecnologias e Stack

Camada	Tecnologia	Versão/Detalhe
Runtime	Node.js	v18+
Framework Backend	Express.js	v4.19
Banco de Dados	PostgreSQL	Driver: pg (node-postgres) v8.12
Autenticação	JWT (jsonwebtoken)	v9.0 + bcrypt v5.1
Frontend	HTML5 + CSS3 + JavaScript	Vanilla JS + Fetch API
Design System	Bootstrap 5	Base + customizações próprias
Containerização	Docker	Ambiente de desenvolvimento
Controle de Versão	Git / GitHub	—

5. Guia de Estilos

Tokens visuais extraídos diretamente das variáveis CSS definidas nos arquivos de estilo da aplicação.

5.1 Paleta de Cores

Cores Primárias

Nome	Hex	Uso
Navy	#0B1730	Fundo escuro principal, textos de título
Navy Light	#162E63	Fundo escuro suave, sidebar gradiente
Blue	#355DAB	Destaque, links, botão confirmar, focus
Blue Dark	#27498E	Hover do Blue
Gold	#FFB300	Ação principal (CTA), botão primário, sidebar ativa
Gold Dark	#E19A00	Hover do Gold

Cores Neutras

Nome	Hex	Uso
Background	#DFE6F3	Fundo das telas de dashboard
Card / Surface	#F8FAFC	Fundo de cards e painéis
Text Primary	#14213D	Texto principal do corpo
Text Muted	#6B7280	Texto secundário e subtítulos
Border	#C8D2E4	Bordas de inputs, cards e divisores
Danger	#EF4444	Erros, botão excluir/cancelar
Success	#16A34A	Sucesso, botão concluir, feedback positivo

Cores de Status — Badges

Status	Fundo	Texto
Pendente	#FFE08A	#6B4F00
Confirmado	#DBEAFE	#1D4ED8
Concluído	#DCFCE7	#15803D
Cancelado	#FEE2E2	#B91C1C

5.2 Tipografia

Fonte	Família / Fallback	Uso	Tamanhos

Oswald	Oswald, Arial Black, sans-serif	Títulos, nome da marca	2.4rem / 2.2rem / 1.5rem
Montserrat	Montserrat, Segoe UI, Arial	Corpo, labels, botões	0.95–0.98rem / labels: 0.9rem

5.3 Componentes Principais

Componente	Descrição Visual
Sidebar	280px, gradiente Navy, título Gold (Oswald), nav-btns com estado ativo em Gold
Botão Primário (CTA)	Fundo Gold (#FFB300), texto Navy, peso 800, hover Gold Dark + translateY(-1px)
Botão Perigo	Fundo Danger (#EF4444), texto branco — logout, cancelar, excluir
Input / Select	Fundo #F9FBFF, borda #C8D2E4, focus: borda Blue + box-shadow azul
Badge de Status	Pill (radius 999px), 0.78rem, peso 800, uppercase — cores por status
Card Item	Fundo branco, borda #C8D2E4, radius 14px, sombra suave
Feedback (erro)	Texto Danger (#EF4444), peso 600
Feedback (sucesso)	Texto Success (#16A34A), peso 600

5.4 Responsividade

Breakpoint	Comportamento
>= 1024px	Layout sidebar + conteúdo lado a lado (desktop)
< 1024px	Sidebar no topo, navegação em linha horizontal (tablet)
< 768px	Painéis com padding reduzido, grids em coluna única
< 480px	Fontes reduzidas, padding mínimo, botões em largura total

6. Requisitos Não Funcionais

ID	Categoria	Requisito
RNF-01	Desempenho	Requisições de listagem devem responder em no máximo 2 segundos para até 50 usuários simultâneos.
RNF-02	Segurança	Senhas com bcrypt (fator 10). JWT com expiração via .env. Rotas protegidas por authMiddleware. Credenciais nunca versionadas.
RNF-03	Disponibilidade	Disponível durante horário de funcionamento (mínimo 12h/dia). Falhas retornam mensagem adequada sem expor detalhes internos.
RNF-04	Manutenibilidade	Arquitetura em camadas com responsabilidades isoladas. Novos endpoints sem alterar módulos existentes.
RNF-05	Usabilidade	Feedback visual para todas as ações. Formulários com validação e mensagens claras. Navegação consistente.
RNF-06	Compatibilidade	Funciona em Chrome, Firefox, Edge e Safari. Interface responsiva para desktop, tablet e mobile.
RNF-07	Escalabilidade	Arquitetura preparada para mensageria (RabbitMQ/Redis) sem refatoração. Banco via pool de conexões (pg.Pool).
RNF-08	Rastreabilidade	Registros com created_at. Erros capturados pelo middleware centralizado com statusCode padronizado.
RNF-09	Confiabilidade	Sistema detecta e rejeita agendamentos conflitantes. Exclusões verificam existência antes de executar (404).
RNF-10	Containerização	Ambiente executável via Docker para consistência entre desenvolvimento e produção.

7. Principais Rotas da API

7.1 Autenticação — /api/auth

Método	Rota	Proteção	Descrição
POST	/api/auth/register	Pública	Cadastro de novo cliente
POST	/api/auth/login	Pública	Login e geração de JWT
GET	/api/auth/me	authMiddleware	Retorna dados do usuário logado

7.2 Serviços — /api/services

Método	Rota	Proteção	Descrição
GET	/api/services	authMiddleware	Lista todos os serviços ativos
POST	/api/services	admin | barbeiro	Cria novo serviço
PUT	/api/services/:id	admin | barbeiro	Edita serviço existente
DELETE	/api/services/:id	admin | barbeiro	Remove serviço

7.3 Agendamentos — /api/appointments

Método	Rota	Proteção	Descrição
GET	/api/appointments	authMiddleware	Lista agendamentos (filtrado por role)
POST	/api/appointments	cliente	Cria novo agendamento
PATCH	/api/appointments/:id/status	admin | barbeiro	Atualiza status do agendamento
DELETE	/api/appointments/:id	authMiddleware	Remove agendamento (dono ou admin)
GET	/api/barbeiros	authMiddleware	Lista barbeiros e admins disponíveis

7.4 Perfis e Permissões

Funcionalidade	Cliente	Barbeiro	Admin
Login / Cadastro	Sim	Sim	Sim
Ver próprios agendamentos	Sim	Sim	Sim
Criar agendamento	Sim	Não	Não
Ver todos os agendamentos	Não	Sim	Sim
Atualizar status	Não	Sim	Sim

Cadastrar / editar serviços	Não	Sim	Sim
Excluir serviços	Não	Sim	Sim

8. Plano de Testes

8.1 Objetivo

Descrever os cenários que serão testados no sistema BarberPro, cobrindo as funcionalidades principais, regras de negócio, segurança e integridade dos dados, garantindo que o sistema se comporte conforme o esperado.

8.2 Escopo

Os testes cobrem as seguintes áreas do sistema:

Autenticação e autorização
Cadastro de usuários
Gerenciamento de serviços
Agendamentos
Controle de acesso por perfil
Validações e tratamento de erros

8.3 Autenticação E Autorização

CT-01 — Login com credenciais válidas
Pré-condição: usuário cadastrado no banco de dados
Ação: enviar POST /api/auth/login com e-mail e senha corretos
Resultado esperado: retorno 200 com token JWT e dados do usuário

CT-02 — Login com senha incorreta
Pré-condição: usuário cadastrado no banco de dados
Ação: enviar POST /api/auth/login com e-mail correto e senha errada
Resultado esperado: retorno 401 com mensagem "Credenciais inválidas."

CT-03 — Login com e-mail não cadastrado
Ação: enviar POST /api/auth/login com e-mail inexistente
Resultado esperado: retorno 401 com mensagem "Credenciais inválidas."

CT-04 — Login com campos ausentes
Ação: enviar POST /api/auth/login sem e-mail ou sem senha
Resultado esperado: retorno 400 com mensagem de campo obrigatório

CT-05 — Acesso a rota protegida sem token
Ação: enviar GET /api/appointments sem header Authorization
Resultado esperado: retorno 401 com mensagem "Token não informado."

CT-06 — Acesso a rota protegida com token inválido
Ação: enviar GET /api/appointments com token malformado
Resultado esperado: retorno 401 com mensagem "Token inválido."

CT-07 — Acesso a rota protegida com token válido
Pré-condição: token JWT válido gerado pelo login
Ação: enviar GET /api/appointments com Authorization: Bearer <token>
Resultado esperado: retorno 200 com lista de agendamentos

8.4 Cadastro De Usuários

CT-08 — Cadastro com dados válidos
Ação: enviar POST /api/auth/register com nome, e-mail e senha válidos
Resultado esperado: retorno 201 com dados do usuário criado (sem senha)

CT-09 — Cadastro com e-mail já existente
Pré-condição: e-mail já cadastrado no banco
Ação: enviar POST /api/auth/register com o mesmo e-mail
Resultado esperado: retorno 409 com mensagem "E-mail já cadastrado."

CT-10 — Cadastro com e-mail inválido
Ação: enviar POST /api/auth/register com e-mail sem formato válido
Resultado esperado: retorno 400 com mensagem "E-mail inválido."

CT-11 — Cadastro com senha curta (menos de 6 caracteres)
Ação: enviar POST /api/auth/register com senha de 5 caracteres
Resultado esperado: retorno 400 com mensagem sobre tamanho mínimo

CT-12 — Cadastro sem nome
Ação: enviar POST /api/auth/register sem o campo nome
Resultado esperado: retorno 400 com mensagem "Nome é obrigatório."

CT-13 — Senha armazenada como hash
Pré-condição: usuário recém-cadastrado
Ação: consultar diretamente o banco de dados
Resultado esperado: campo senha_hash contém hash bcrypt, não texto puro

8.5 Gerenciamento De Serviços

CT-14 — Listar serviços autenticado
Pré-condição: token JWT válido
Ação: enviar GET /api/services
Resultado esperado: retorno 200 com array de serviços ativos

CT-15 — Criar serviço como admin
Pré-condição: token JWT de usuário com role admin ou barbeiro
Ação: enviar POST /api/services com nome, preço e duração válidos
Resultado esperado: retorno 201 com dados do serviço criado

CT-16 — Criar serviço como cliente (sem permissão)
Pré-condição: token JWT de usuário com role cliente
Ação: enviar POST /api/services
Resultado esperado: retorno 403 com mensagem "Acesso negado."

CT-17 — Criar serviço com nome muito curto
Ação: enviar POST /api/services com nome de 2 caracteres
Resultado esperado: retorno 400 com mensagem sobre tamanho mínimo

CT-18 — Criar serviço com preço inválido (zero ou negativo)
Ação: enviar POST /api/services com preco = 0
Resultado esperado: retorno 400 com mensagem "Preço inválido."

CT-19 — Criar serviço com duração inválida
Ação: enviar POST /api/services com duracao_min = 0 ou negativo
Resultado esperado: retorno 400 com mensagem "Duração inválida."

CT-20 — Editar serviço existente como admin
Pré-condição: serviço existente, token de admin/barbeiro
Ação: enviar PUT /api/services/:id com dados válidos
Resultado esperado: retorno 200 com dados atualizados

CT-21 — Editar serviço inexistente
Pré-condição: token de admin/barbeiro
Ação: enviar PUT /api/services/99999
Resultado esperado: retorno 404 com mensagem "Serviço não encontrado."

CT-22 — Excluir serviço como admin
Pré-condição: serviço existente, token de admin/barbeiro
Ação: enviar DELETE /api/services/:id
Resultado esperado: retorno 200 com dados do serviço removido

CT-23 — Excluir serviço inexistente
Pré-condição: token de admin/barbeiro
Ação: enviar DELETE /api/services/99999
Resultado esperado: retorno 404 com mensagem "Serviço não encontrado."

8.6 Agendamentos

CT-24 — Criar agendamento válido como cliente
Pré-condição: token de cliente, barbeiro e serviço existentes
Ação: enviar POST /api/appointments com cliente_id, barbeiro_id, service_id e data_hora futura válidos
Resultado esperado: retorno 201 com dados do agendamento criado

CT-25 — Criar agendamento para data no passado
Pré-condição: token de cliente
Ação: enviar POST /api/appointments com data_hora anterior ao momento atual
Resultado esperado: retorno 400 com mensagem sobre data passada

CT-26 — Criar agendamento com conflito de horário
Pré-condição: barbeiro já possui agendamento no mesmo horário
Ação: enviar POST /api/appointments com mesmo barbeiro e horário sobreposto
Resultado esperado: retorno 409 com mensagem sobre conflito de horário

CT-27 — Criar agendamento sem barbeiro informado
Pré-condição: token de cliente
Ação: enviar POST /api/appointments sem o campo barbeiro_id
Resultado esperado: retorno 400 com mensagem "Selecione um barbeiro."

CT-28 — Criar agendamento com serviço inexistente
Pré-condição: token de cliente
Ação: enviar POST /api/appointments com service_id inválido
Resultado esperado: retorno 404 com mensagem "Serviço não encontrado."

CT-29 — Criar agendamento como admin (sem permissão de criar)
Pré-condição: token de admin
Ação: enviar POST /api/appointments
Resultado esperado: retorno 403 com mensagem "Acesso negado."

CT-30 — Listar agendamentos como cliente
Pré-condição: token de cliente com agendamentos cadastrados
Ação: enviar GET /api/appointments
Resultado esperado: retorno 200 com apenas os agendamentos do próprio cliente

CT-31 — Listar agendamentos como admin
Pré-condição: token de admin
Ação: enviar GET /api/appointments
Resultado esperado: retorno 200 com todos os agendamentos do sistema

CT-32 — Atualizar status de agendamento como admin
Pré-condição: agendamento existente, token de admin/barbeiro
Ação: enviar PATCH /api/appointments/:id/status com status "confirmado"

Resultado esperado: retorno 200 com agendamento atualizado

CT-33 — Atualizar status com valor inválido
Pré-condição: token de admin/barbeiro
Ação: enviar PATCH /api/appointments/:id/status com status "invalido"
Resultado esperado: retorno 400 com mensagem "Status inválido."

CT-34 — Atualizar status como cliente (sem permissão)
Pré-condição: token de cliente
Ação: enviar PATCH /api/appointments/:id/status
Resultado esperado: retorno 403 com mensagem "Acesso negado."

CT-35 — Cancelar próprio agendamento como cliente
Pré-condição: token do cliente dono do agendamento
Ação: enviar DELETE /api/appointments/:id
Resultado esperado: retorno 200 com dados do agendamento removido

CT-36 — Tentar cancelar agendamento de outro cliente
Pré-condição: token de cliente diferente do dono do agendamento
Ação: enviar DELETE /api/appointments/:id
Resultado esperado: retorno 403 com mensagem de permissão negada

CT-37 — Cancelar agendamento inexistente
Pré-condição: token válido
Ação: enviar DELETE /api/appointments/99999
Resultado esperado: retorno 404 com mensagem "Agendamento não encontrado."

8.7 Controle De Acesso Por Perfil

CT-38 — Cliente não acessa rota exclusiva de admin
Pré-condição: token de cliente
Ação: enviar POST /api/services (rota restrita a admin/barbeiro)
Resultado esperado: retorno 403 com mensagem "Acesso negado."

CT-39 — Admin acessa todas as rotas protegidas
Pré-condição: token de admin
Ação: acessar GET /api/appointments, GET /api/services, GET /api/barbeiros
Resultado esperado: retorno 200 em todas as rotas

CT-40 — Listar barbeiros autenticado
Pré-condição: token JWT válido
Ação: enviar GET /api/barbeiros
Resultado esperado: retorno 200 com lista de usuários com role admin ou barbeiro

8.8 Validações E Tratamento De Erros

CT-41 — Erro interno retorna mensagem padronizada
Ação: simular erro no banco de dados
Resultado esperado: retorno 500 com mensagem padronizada, sem stack trace exposto

CT-42 — Rota inexistente
Ação: acessar GET /api/rota-que-nao-existe
Resultado esperado: retorno 404 (tratado pelo Express)

CT-43 — Body com tipo errado (ex: preço como string)
Ação: enviar POST /api/services com preco: "vinte reais"
Resultado esperado: retorno 400 com mensagem "Preço inválido."

8.9 Resumo dos Cenários

Total de cenários mapeados: 43

Por área:		
Autenticação e autorização	7 cenários	(CT-01 a CT-07)
Cadastro de usuários	6 cenários	(CT-08 a CT-13)
Gerenciamento de serviços	10 cenários	(CT-14 a CT-23)
Agendamentos	14 cenários	(CT-24 a CT-37)
Controle de acesso	3 cenários	(CT-38 a CT-40)
Validações e erros	3 cenários	(CT-41 a CT-43)

Tipos de teste previstos:	
Teste funcional	validar comportamento das funcionalidades
Teste de segurança	validar autenticação, autorização e senhas
Teste de validação	validar regras de entrada de dados
Teste de integridade	validar regras de negócio (conflito de horário, permissões)
Teste de erro	validar respostas HTTP para casos de falha EOF

9. Como Executar o Projeto

9.1 Pré-requisitos

- Node.js v18+ instalado
- PostgreSQL instalado e em execução
- Git instalado
- Docker (opcional, para ambiente containerizado)

9.2 Passo a Passo

1. Clone o repositório

```
git clone https://github.com/GabrielDaSilvaCoelho/BarbeariaTerminal.git
cd BarbeariaTerminal-main
```

2. Instale as dependências

```
npm install
```

3. Configure o .env

```
PORT=3000
DB_HOST=localhost
DB_PORT=5432
DB_NAME=barberpro
DB_USER=postgres
DB_PASSWORD=sua_senha
JWT_SECRET=sua_chave_jwt
JWT_EXPIRES_IN=7d
```

4. Crie o banco e execute o SQL

```
CREATE DATABASE barberpro;
-- Execute o arquivo database.sql
```

5. Popule e inicie

```
npm run seed
npm start
```

9.3 Usuários para Demonstração

Perfil	E-mail	Senha
Administrador	admin@barberpro.com	123456
Cliente	cliente@barberpro.com	123456